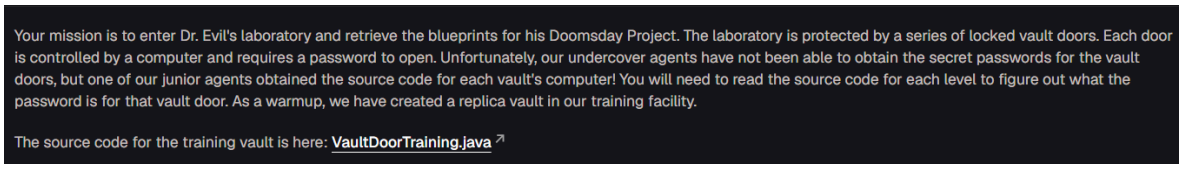
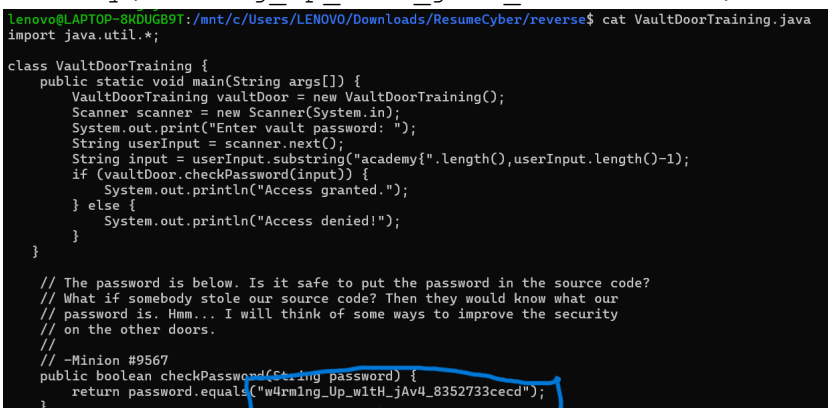


Challenge 1 : Vault-door-training

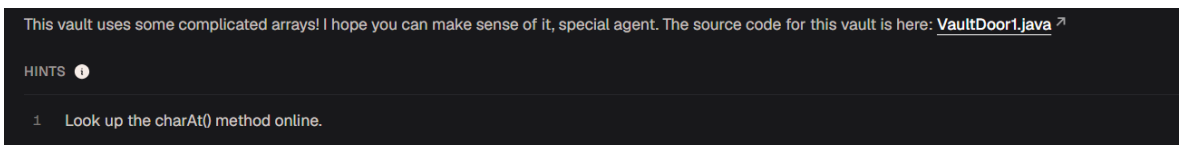


File Bernama : VaultDoorTraining.java

1. Kita baca filenya dengan cat dan ada yang janggal pada codingannya.
2. Dan yap ketemu flagnya yakni
   `academy{w4rm1ng_Up_w1tH_jAv4_8352733cecd}`



Challenge 2 : vault-door-1



File bernama : VaultDoor1.java

1. Kita baca file dengan cat seperti biasanya dan ada hal janggal seperti output flag
2. Kita bisa aja menyusunnya secara manual untuk menemukan flagnya, akan tetapi kita disini bakal menggunakan python untuk menyusunnya secara rapi menggunakan **solve.py**

```
public boolean checkPassword(String password) {
    return password.length() == 32 &&
           password.charAt(0)  == 'd' &&
           password.charAt(29) == 'c' &&
           password.charAt(4)  == 'r' &&
           password.charAt(2)  == '5' &&
           password.charAt(23) == 'r' &&
           password.charAt(3)  == 'c' &&
           password.charAt(17) == '4' &&
           password.charAt(1)  == '3' &&
           password.charAt(7)  == 'b' &&
           password.charAt(10) == '_' &&
           password.charAt(5)  == '4' &&
           password.charAt(9)  == '3' &&
           password.charAt(11) == 't' &&
           password.charAt(15) == 'c' &&
           password.charAt(8)  == 'l' &&
           password.charAt(12) == 'H' &&
           password.charAt(20) == 'c' &&
           password.charAt(14) == '_' &&
           password.charAt(6)  == 'm' &&
           password.charAt(24) == '5' &&
           password.charAt(18) == 'r' &&
           password.charAt(13) == '3' &&
           password.charAt(19) == '4' &&
           password.charAt(21) == 'T' &&
           password.charAt(16) == 'H' &&
           password.charAt(27) == '2' &&
           password.charAt(30) == 'a' &&
           password.charAt(25) == '_' &&
           password.charAt(22) == '3' &&
           password.charAt(28) == '4' &&
           password.charAt(26) == '7' &&
           password.charAt(31) == '5';
}
```

3. Kita jalankan **solve.py** dan yap ketemu flagnya yakni `academy{d35cr4mbl3_tH3_cH4r4cT3r5_724ca5}`

```
}lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ nano solve.py
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ python3 solve.py
Password: d35cr4mbl3_tH3_cH4r4cT3r5_724ca5
```

Challenge 3 : vault-door-3

This vault uses for-loops and byte arrays. The source code for this vault is here: VaultDoor3.java

HINTS

1 Make a table that contains each value of the loop variables and the corresponding buffer index that it writes to.

File bernama : VaultDoor3.java

1. Kita baca filenya dengan cat seperti biasanya dan menemukan output janggal sebagai berikut

```
            buffer[i] = password.charAt(i);
        }
        String s = new String(buffer);
        return s.equals("jU5t_a_sna_3lpm12ga4b_u_4_mfrc46");
    }
```

2. Lalu kita masukkan output janggal tersebut untuk kita decode dengan source code **solve1.py**
3. Kita jalankan codenya dan yap kita ketemu flagnya yakni `academy{jU5t_a_s1mpl3_an4gr4m_4_u_bfac26}`

```
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ nano solve1.py
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ python3 solve1.py
Password: jU5t_a_s1mpl3_an4gr4m_4_u_bfac26
```

Challenge 4 : vault-door-4

Reverse Engineering Medium
by Mark E. Haase · picoCTF 2019

This vault uses ASCII encoding for the password. The source code for this vault is here: VaultDoor4.java

HINTS

1 Use a search engine to find an "ASCII table"

2 You will also need to know the difference between octal, decimal, and hexadecimal numbers.

File bernama : VaultDoor4.java

1. Kita baca lagi dengan cat dan menemukan output janggal

```
public boolean checkPassword(String password) {
    byte[] passBytes = password.getBytes();
    byte[] myBytes = {
        106 , 85  , 53  , 116 , 95  , 52  , 95  , 98  ,
        0x55, 0x6e, 0x43, 0x68, 0x5f, 0x30, 0x66, 0x5f,
        0142, 0131, 0164, 063 , 0163, 0137, 066 , 0141,
        '8' , 'a' , '9' , 'e' , '9' , '6' , 'b' , 'c' ,
    };
```

2. Karena di soal ada ketentuan ascii kita lakukan decode dengan metode **ASCII** (secara manual : https://chat.deepseek.com/share/yknm1tgxjrneit895f)
3. Kita bisa menggunakan python untuk decode **ASCII** dengan **solve2.py**
4. Kita jalankan dan yap kita ketemu flagnya yakni `academy{jU5t_4_bUnCh_0f_bYt3s_6a8a9e96bc}`

```
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ nano solve2.py
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ python3 solve2.py
Password: jU5t_4_bUnCh_0f_bYt3s_6a8a9e96bc
```

Challenge 5 : vault-door-5

In the last challenge, you mastered octal (base 8), decimal (base 10), and hexadecimal (base 16) numbers, but this vault door uses a different change of base as well as URL encoding! The source code for this vault is here: VaultDoor5.java

HINTS

1 You may find an encoder/decoder tool helpful, such as https://encoding.tools/

2 Read the wikipedia articles on URL encoding and base 64 encoding to understand how they work and what the results look like.

File bernama : VaultDoor5.java

1. Kita lakukan cat lagi dan yap ketemu output janggal yakni sebuah **strings** yang kita bisa decode menjadi **base64**

```
public boolean checkPassword(String password) {
    String urlEncoded = urlEncode(password.getBytes());
    String base64Encoded = base64Encode(urlEncoded.getBytes());
    String expected = "JTYzJTMwJTZlJTc2JTMzJTcyJTc0JTMxJTZlJTY3JTVm"
                    + "JTY2JTcyJTMwJTZkJTVmJTYyJTYxJTM1JTY1JTVmJTM2"
                    + "JTM0JTVmJTMzJTYzJTYzJTM4JTYxJTM5JTY0JTM2";
    return base64Encoded.equals(expected);
}
```

2. Kita jadikan satu yang ada di dalam **string expected,** lalu kita lakukan **decode base64** dan **decode url** sesuai outputnya secara urutan. Jadi **decode base64** dulu baru **decode url**. Dan yap kita ketemu flagnya yakni `academy{c0nv3rt1ng_fr0m_ba5e_64_3cc8a9d6}`

```
}lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ echo "JTYzJTMwJTZlJTc2JTMzJTcyJTc0JTMxJTZlJTY3JTVmJTY2JTcyJTMwJTZkJTV
mJTYyJTYxJTM1JTY1JTVmJTM2JTM0JTVmJTMzJTYzJTYzJTM4JTYxJTM5JTY0JTM2" | base64 -d
%63%30%6e%76%33%72%74%31%6e%67%5f%66%72%30%6d%5f%62%61%35%65%5f%36%34%5f%33%63%63%38%61%39%64%36lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downl
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ echo "%63%30%6e%76%33%72%74%31%6e%67%5f%66%72%30%6d%5f%62%61%35%65%5f%
36%34%5f%33%63%63%38%61%39%64%36" | sed 's/%//g' | xxd -r -p
c0nv3rt1ng_fr0m_ba5e_64_3cc8a9d6lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$
```

Challenge 6 : vault-door-6

This vault uses an XOR encryption scheme. The source code for this vault is here: VaultDoor6.java

HINTS

1. If X ^ Y = Z, then Z ^ Y = X. Write a program that decrypts the flag based on this fact.

File bernama : VaultDoor6.java

1. Kita baca lagi source codenya dengan cat dan menemukan hal janggal yakni pada **check password** dan kumpulan **bytes**

```
public boolean checkPassword(String password) {
    if (password.length() != 32) {
        return false;
    }
    byte[] passBytes = password.getBytes();
    byte[] myBytes = {
        0x3b, 0x65, 0x21, 0xa , 0x38, 0x0 , 0x36, 0x1d,
        0xa , 0x3d, 0x61, 0x27, 0x11, 0x66, 0x27, 0xa ,
        0x21, 0x1d, 0x61, 0x3b, 0xa , 0x2d, 0x65, 0x27,
        0xa , 0x6d, 0x31, 0x65, 0x63, 0x36, 0x66, 0x60,
    };
    for (int i=0; i<32; i++) {
        if (((passBytes[i] ^ 0x55) - myBytes[i]) != 0) {
            return false;
        }
    }
    return true;
}
```

2. Kita buat file **solve3.py** untuk menemukan flagnya dan yap kita ketemu flagnya yakni `academy{n0t_mUcH_h4rD3r_tH4n_x0r_8d06c35}`

```
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ nano solve3.py
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ python3 solve3.py
Password: n0t_mUcH_h4rD3r_tH4n_x0r_8d06c35
```

Challenge 7 : vault-door-7

Reverse Engineering Hard

by Mark E. Haase · picoCTF 2019

This vault uses bit shifts to convert a password string into an array of integers. Hurry, agent, we are running out of time to stop Dr. Evil's nefarious plans! The source code for this vault is here: VaultDoor7.java

HINTS

1. Use a decimal/hexadecimal converter such as this one: https://www.mathsisfun.com/binary-decimal-hexadecimal-converter.html
2. You will also need to consult an ASCII table such as this one: https://www.asciitable.com/

File bernama : VaultDoor7.java

1. Kita lihat filenya dengan cat dan menemukan hal janggal

Di dalam fungsi `checkPassword`, password 32 karakter dikelompokkan menjadi 8 kelompok (masing-masing 4 karakter) dan diubah menjadi array integer (`int`).

java Copy Download

```
x[i] = hexBytes[i*4]   << 24
     | hexBytes[i*4+1] << 16
     | hexBytes[i*4+2] << 8
     | hexBytes[i*4+3];
```

Ini berarti 4 byte (4 karakter ASCII) digabungkan menjadi satu angka 32-bit.
Untuk membaliknya, kita cukup mengubah setiap angka desimal menjadi **format Hexadesimal 8 digit** (karena 1 byte = 2 digit hex, 4 byte = 8 digit hex).

Setelah menjadi hex, kita ubah setiap 2 digit hex tersebut menjadi karakter ASCII.

2. Lalu kita buat file solve python **solve4.py** dan kita jalankan source codenya dan yap kita ketemu flagnya `academy{A_b1t_0f_b1t_sh1fTiNg_8b5aaa66d1}`

```
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ nano solve4.py
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ python3 solve4.py
Password: A_b1t_0f_b1t_sh1fTiNg_8b5aaa66d1
lenovo@LAPTOP-8KDUGB9T:/mnt/c/Users/LENOVO/Downloads/ResumeCyber/reverse$ cat VaultDoor8.java
```

Challenge 8 : vault-door-8

Apparently Dr. Evil's minions knew that our agency was making copies of their source code, because they intentionally sabotaged this source code in order to make it harder for our agents to analyze and crack into! The result is a quite mess, but I trust that my best special agent will find a way to solve it. The source code for this vault is here: VaultDoor8.java

HINTS

1. Clean up the source code so that you can read it and understand what is going on.
2. Draw a diagram to illustrate which bits are being switched in the scramble() method, then figure out a sequence of bit switches to undo it. You should be able to reuse the switchBits() method as is.

File bernama : VaultDoor8.java

1. Kita lihat lagi isi filenya dan amati hal janggal

Jika kita rapikan, fungsi `scramble` memodifikasi setiap karakter dengan menukar posisi bit tertentu secara berurutan:

1. Tukar bit 1 dan 2
2. Tukar bit 0 dan 3
3. Tukar bit 5 dan 6
4. Tukar bit 4 dan 7
5. Tukar bit 0 dan 1
6. Tukar bit 3 dan 4
7. Tukar bit 2 dan 5
8. Tukar bit 6 dan 7

Karena setiap operasi penukaran bit (`switchBits`) bersifat **simetris** (menukar A dengan B sama dengan menukar B dengan A), maka untuk membalik seluruh proses (*unscramble*), kita cukup **menjalankan urutan penukaran tersebut secara terbalik** (dari langkah 8 kembali ke langkah 1).

Target `expected` yang harus dicapai adalah array heksadesimal ini:

java Copy Download

```
char[] expected = {
    0xF4, 0xC0, 0x97, 0xF0, 0x77, 0x97, 0xC0, 0xE4,
    0xF0, 0x77, 0xA4, 0xD0, 0xC5, 0x77, 0xF4, 0x86,
    0xD0, 0xA5, 0x45, 0x96, 0x27, 0xB5, 0x77, 0x95,
    0x94, 0xC1, 0xE1, 0xD2, 0xE1, 0xE1, 0xD0, 0x95
};
```

2. Kita buat file solvenya dengan **solve5.py** dan kita jalankan source codenya dan yap kita menemukan flagnya yakni

academy{s0m3_m0r3_b1t_sh1fTiNg_ea469661e}

